



Morning Brief

Ludovici · Tuesday, June 9, 2026

In this edition

1. AI governance & policy · 2. Cybersecurity & threats · 3. Project status · 4. Five project ideas · 5. Training — Python: vectorization & broadcasting · 6. Training — R: `purrr` + list-columns (many models at once) · 7. Training — Data-centric: data contracts · 8. Training — AI method of the day: Denoising Diffusion Probabilistic Models (DDPM)

Prepared for Stephanie Ludovici

Email and calendar sections will appear here once Gmail and Google Calendar are connected. This edition covers news, project status, ideas, and training.

1. AI governance & policy

- **NIST opens an autonomous-agent standards effort.** Building on the AI RMF, NIST has moved to develop standards and evaluation methods for autonomous AI agents — systems that take actions with limited human oversight — signaling that agentic AI is the next governance frontier. Directly relevant to your v3.0 agentic-overlay thread. [NIST AI RMF](#)
- **NIST concept note: AI RMF Profile for Critical Infrastructure (Apr 7, 2026).** A forthcoming profile will guide critical-infrastructure operators on risk practices for AI-enabled capabilities — another sector-specific overlay on the RMF spine. [NIST AI RMF](#)
- **EU AI Act — the August 2, 2026 milestone is now under two months out.** The bulk of the Act's obligations (high-risk Annex III systems, Article 50 transparency duties) begin to apply Aug 2,

2026; GPAI obligations have applied since Aug 2, 2025. [EU AI Act timeline](#)

- **EU moves to simplify the rulebook (May 7, 2026).** The Council and Parliament agreed to streamline parts of the AI Act and related digital rules — worth watching for changes to compliance timing and scope. [Consilium press release](#)

2. Cybersecurity & threats

- **CISA KEV additions (Jun 2–3, 2026).** New actively-exploited entries include CVE-2022-0492 (Linux kernel privilege escalation) and CVE-2025-48595 (Android Framework integer overflow); BOD 22-01 sets federal remediation deadlines. Worth a glance if any touch your stack. [CISA KEV catalog](#)
- **Exploit-before-patch is now common.** Recent incident-response data finds ~28% of CVEs exploited within 24 hours of disclosure, with some intrusions spreading laterally in under a minute — patch windows are collapsing. [Unit 42 IR Report](#)
- **Extortion is shifting away from encryption.** Encryption appeared in ~78% of extortion cases in 2025 (down from ~90% + in 2021–2024); data-theft-only extortion is rising as actors cut operational complexity. [BlackFog: State of Ransomware 2026](#)
- **Agentic AI in the attack chain.** Threat-intel forecasts describe AI handling reconnaissance, scanning, and even negotiation with reduced human oversight — the offensive mirror of your defensive agentic-governance work. [WEF Global Cybersecurity Outlook 2026](#)

3. Project status

Scans all projects under your `Projects` folder and leads with the most active; today that's the AI Governance Framework.

Where it stands. The framework (risk-proportionate three-tier model, 12 sections, 17 artifact templates, NIST AI RMF / SP 800-53 / ICD 505 crosswalks, plus Python reference implementations for risk scoring, drift detection, fairness, model cards, and inventory) is shipped at **v2.14 (June 2026)** across the whitepaper, framework, calibration study, implementation playbook, modeling-and-simulation, and peer-review deliverables.

Where you left off (v3.0 tracker). Four forward-looking threads are parked in `V3.0-ROADMAP-FOLLOWUP.md`: 1. **Categorical pre-screening** — already prototyped and validated (`prescreen.py`, 100%

consistency, 0 mismatches). Decision pending: ship it in v3.0 and define the production attribute taxonomy. 2. **Agentic-AI governance overlay** — evidence currently rests on OWASP Agentic Top 10 (2026) *only*; NIST/academic agent-eval guidance did not survive verification. 3. **GenAI groundedness/hallucination overlay** — needs a numeric pass/fail threshold decision and a domain validation study for a local checker (MiniCheck) on classified-domain text. 4. **Principles-validation & modality overlays** — named in §12.4 but not yet developed.

Suggested next action. The external news above (NIST autonomous-agent standards effort) is a prompt to act on thread #2: re-verify the agentic evidence base beyond OWASP now that NIST has opened agent standards work, so the overlay rests on more than a single source before any control is drafted.

4. Five project ideas

Full write-ups (problem statement, objective, method/approach, estimated duration, sources) are archived for later review in `project-ideas/2026-06-09-ideas.md`. Today's set:

1. **Physics-informed neural network for subsurface permeability inversion** — Julia/SciML; recover a permeability field from sparse sensors via an inverse PDE. *~4–6 weeks*.
2. **E(3)-equivariant GNN surrogate for materials & catalyst screening** — Python/e3nn; symmetry-respecting energy+force model on a single GPU. *~5–7 weeks*.
3. **Conformal prediction for distribution-free scientific uncertainty** — R (+ Python MAPIE); guaranteed-coverage prediction sets under shift. *~3–4 weeks*.
4. **Fourier Neural Operator surrogate for PDE forecasting** — Python `neuraloperator`; resolution-invariant solver surrogate for fluid/climate PDEs. *~5–6 weeks*.
5. **Bayesian hierarchical model for ecological & epidemiological counts** — R/Stan (+ Julia `Turing.jl`); partial-pooling model for overdispersed, zero-inflated counts. *~4–5 weeks*.

5. Training — Python: vectorization & broadcasting

Python `for` loops over numerical data are slow because every iteration pays interpreter overhead and boxes each scalar as an object. NumPy

moves the loop into compiled C over contiguous memory and applies SIMD — often 10–100× faster. **Broadcasting** is the rule set that lets NumPy combine arrays of different shapes without copying: dimensions are aligned from the right, and any axis of length 1 is stretched to match.

A classic worked example — pairwise squared Euclidean distances between two point sets:

```
import numpy as np

def pairwise_sq_dists_loop(a, b):
    # Naive triple loop: correct, but pays Python overhead
    n, m = a.shape[0], b.shape[0]
    out = np.empty((n, m)) # Pre-allocate the result grid
    for i in range(n):
        for j in range(m):
            diff = a[i] - b[j] # Vector subtract
            out[i, j] = diff @ diff # Dot product == ||diff||^2
    return out

def pairwise_sq_dists_broadcast(a, b):
    # Broadcasting: insert axes so shapes align as (n, 1, d) and (1, m, d)
    diff = a[:, None, :] - b[None, :, :] # Result shape (n, m, d)
    return np.einsum("nmd,nmd->nm", diff, diff) # Sum of squares

def pairwise_sq_dists_matmul(a, b):
    # Identity ||x - y||^2 = ||x||^2 + ||y||^2 - 2 x.y available
    # intermediate entirely — the memory-efficient choice
    a2 = np.einsum("nd,nd->n", a, a)[:, None] # (n, 1)
    b2 = np.einsum("md,md->m", b, b)[None, :] # (1, m)
    return a2 + b2 - 2.0 * (a @ b.T) # Broadcast
```

The trap: the broadcast version builds an (n, m, d) intermediate. At $n = m = 20000, d = 100$ that is ~3 TB — it will crash. The `matmul` identity only ever holds (n, m) , so prefer it for large inputs (clip tiny negatives from floating-point error with `np.maximum(out, 0)` before `sqrt`). Lesson: vectorize for speed, but always check the *shape of the largest intermediate*.

Reference: [NumPy — Broadcasting](#).

6. Training — R: purrr + list-columns (many models at once)

A powerful idiom in modern R is to keep *data frames of data frames* and map functions over them, instead of copy-pasting analysis code per group. `tidyr::nest()` collapses each group into a list-column; `purrr::map()` applies a model to each; `broom` tidies the results back into a flat table. It is composable, type-stable, and trivially parallelizable with `furrr`.

```
library(dplyr)
library(tidyr)
library(purrr)
library(broom)

# Fit one linear model per species, then extract slopes and
results <- iris |>
  nest(data = -Species) |>
  mutate(
    model    = map(data, \(df) lm(Sepal.Length ~ Petal.Length, data = df)),
    coefs     = map(model, tidy),
    quality   = map(model, glance)
  )

slopes <- results |>
  select(Species, coefs) |>
  unnest(coefs) |>
  filter(term == "Petal.Length")
```

`slopes` now holds one tidy row per species with estimate, standard error, and p-value — no loops, no fragile copy-paste. Swap `map` for `furrr::future_map` to run the fits in parallel. Scale this pattern to bootstrap resamples, cross-validation folds, or hyperparameter grids.

Reference: Wickham & Grolemund, *R for Data Science* (2e), "Many models" — <https://r4ds.hadley.nz/>.

7. Training — Data-centric: data contracts

A **data contract** is a versioned, enforceable agreement at the boundary between a data producer and its consumers. It specifies the **schema** (fields and types), the **semantics** (what each field means, units, allowed values), **quality expectations** (e.g., completeness, validity, freshness), **SLAs** (update cadence, availability), and **ownership** (who is accountable). The point is to *shift data-quality enforcement left* — onto the producer at the point of publish — instead of letting consumers discover breakage downstream.

Why it matters for governance: a contract turns vague data ownership into a testable artifact. It maps cleanly onto DAMA-DMBOK data-quality dimensions (completeness, validity, consistency, timeliness) and onto stewardship accountability, and it gives change management a gate — a breaking schema change must bump the contract version and notify consumers.

A minimal contract, enforced in CI before data is published:

```
dataset: ai_use_case_registry
version: 1.2.0
owner: ai-governance-board
fields:
  - {name: use_case_id, type: string, required: true, unique: true}
  - {name: risk_tier, type: int, required: true, allow_null: false}
  - {name: updated_at, type: timestamp, required: true}
quality:
  - rule: freshness, column: updated_at, max_age: 24h
  - rule: not_null, columns: [use_case_id, risk_tier]
sla: {availability: "99%", update_cadence: daily}
```

Validate each batch against this with a schema/expectations library (Pydantic, JSON Schema, or Great Expectations); on violation, **quarantine** the batch rather than publish bad data. This is the same philosophy your framework already applies to model artifacts — extended to the data tier.

Reference: DAMA International, *DAMA-DMBOK: Data Management Body of Knowledge* (2nd ed.), Data Quality chapter.

8. Training — AI method of the day: Denoising Diffusion Probabilistic Models (DDPM)

Diffusion models are the engine behind much of modern generative AI for images, and increasingly for science (de novo molecule and material design, protein structure, and PDE-constrained inverse problems). The idea is to learn to **reverse a gradual noising process**.

- **Forward process:** start from a real sample x_0 and add a little Gaussian noise over T steps along a fixed variance schedule $\beta_1 \dots \beta_T$, until x_T is essentially pure noise. Because the steps are Gaussian, you can jump to any step in closed form: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon$, where $\epsilon \sim N(0, I)$ and $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$.
- **Reverse process:** train a neural network to undo one noising step. The training objective simplifies elegantly to **predicting**

the noise ϵ that was added at a random timestep — a plain mean-squared-error loss: $L = E \|\epsilon - \epsilon_\theta(x_t, t)\|^2$.

- **Sampling:** start from pure noise x_T and iteratively denoise with ϵ_θ down to a clean sample.

Practical notes: DDPM sampling is slow (hundreds–thousands of steps); **DDIM** and distillation cut this to tens of steps. The continuous-time / score-based SDE view (Song et al., 2021) unifies diffusion with score matching and is the basis for guided and conditional generation — the hook for scientific inverse problems, where you condition on measurements.

References: Ho, Jain & Abbeel, "Denoising Diffusion Probabilistic Models," *NeurIPS* (2020), arXiv:2006.11239; Sohl-Dickstein et al. (2015), arXiv:1503.03585.

Companion notebook: notebooks/2026-06-09-gaussian-process-demo.ipynb — a runnable Gaussian-process regression demo (markdown + code + visualization) illustrating the **M&S** → **model** → **T&E** pattern these briefs use to *show* a method, not just describe it. Runs on NumPy + matplotlib alone.

Novelty ledgers updated: project-ideas/_ledger.md, training-ledger.md. Reputable sources only (government, academia, recognized industry). — Prepared for Stephanie Ludovici.

Attached: brief (.pdf) + companion notebook (.ipynb) + project briefs (.docx)
Morning Brief · Ludovici · Tuesday, June 9, 2026
By Stephanie Ludovici